CPU Quest – gra platformowa edukacyjna

Sprawozdanie z projektu

Michał Kossakowski 197589, Hubert Kowalski 197811

Organizacja Systemów Komputerowych – laboratorium

12 marca 2026

Spis treści

1 Sformułowanie zadania i przyjęte założenia **2**
1.1 Cel projektu 2
1.2 Założenia szczegółowe 2

2 Opis przyjętych rozwiązań programowych **2**
2.1 Wybór technologii 2
2.2 Struktura programu 3
2.3 Obsługa zdarzeń 3
2.4 Mechanika ruchu i kolizji 3
2.5 System monet i rejestrów 4
2.6 System skórek i sklep 4
2.7 System poziomów i odblokowywanie 4
2.8 Pasek ciekawostek 4

3 Dyskusja osiągniętych wyników **4**
3.1 Zalety aplikacji 4
3.2 Wady i ograniczenia 5
3.3 Możliwości rozwoju 5
3.4 Podsumowanie 5

1 Sformułowanie zadania i przyjęte założenia

1.1 Cel projektu

Celem projektu było stworzenie aplikacji z graficznym interfejsem użytkownika (GUI) w języku Python, wykorzystującej monitor, klawiaturę i mysz do interakcji. Aplikacja miała mieć charakter edukacyjny, nawiązujący do tematyki przedmiotu Organizacja Systemów Komputerowych. Wybrano realizację w postaci gry platformowej, w której gracz (symbolizujący proces) zbiera monety (dane) i rejestry, unikając dronów (przerwań). Gra umożliwia konfigurację wyglądu (skórki postaci), wybór poziomów oraz posiada sklep, w którym za zebrane monety można odblokowywać nowe skórki. Dodatkowo w interfejsie wyświetlany jest bieżący czas (zegar RTC) oraz pasek z ciekawostkami z zakresu architektury komputerów.

1.2 Założenia szczegółowe

- **Mechanika gry**: gracz porusza się po planszy (strzałki/WASD), skacze (spacja), podlega grawitacji. Może zbierać żółte monety (dodają 1 monetę) oraz niebieskie rejestry (dodają 5 monet). Czerwone drony poruszają się losowo i po kontakcie zadają 50 punktów obrażeń (HP). Flaga I (przełączana klawiszem I) decyduje, czy drony są aktywne.
- **Poziomy**: gra zawiera trzy poziomy o rosnącym stopniu trudności. Po ukończeniu poziomu (zebraniu wszystkich monet i rejestrów) odblokowuje się następny.
- **Postacie**: dostępne są cztery skórki (Wojska Lądowe, Żandarmeria, WOC, GROM). Podstawowa jest darmowa, pozostałe można kupić w sklepie za monety (odpowiednio 50, 100, 200). W menu wyboru postaci można zmienić aktualnie używaną skórkę.
- **Sklep**: umożliwia zakup nowych skórek. Po zakupie skórka staje się dostępna w wyborze postaci.
- **Kara za śmierć**: gdy HP spadnie do 0, gracz wraca do menu głównego i traci 50 monet (ale nie mniej niż 0).
- **Interfejs**: na ekranie gry wyświetlane są: liczba monet, aktualne HP, bieżący czas, tryb procesora (Real/Protected), flaga I, licznik rozkazów (PC) oraz numer poziomu. Na dole ekranu znajduje się pasek z ciekawostkami (zmieniającymi się co 3 sekundy).
- **Grafika**: wszystkie elementy graficzne (postacie, moneta, rejestry, dron, platformy, tła) zostały wykonane samodzielnie w programie graficznym i są przechowywane w folderze `zdjecia`.

2 Opis przyjętych rozwiązań programowych

2.1 Wybór technologii

Do implementacji wykorzystano język Python z biblioteką **Pygame**. Jest to popularna biblioteka do tworzenia gier 2D, oferująca łatwą obsługę grafiki, zdarzeń oraz dźwięku. Wybór podyktowany był prostotą oraz dostępnością na platformę PC.

2.2 Struktura programu

Program został zorganizowany w sposób obiektowy. Główne klasy to:

- `Game` – zarządza stanem gry, menu, poziomami, sklepem, wyborem postaci.
- `Player` – reprezentuje gracza, obsługuje ruch, grawitację i kolizje.
- `Platform`, `Coin`, `Register`, `Interrupt` – klasy sprite'ów dla poszczególnych elementów.
- `Button` – klasa przycisku obsługująca zdarzenia myszy i opcjonalną miniaturkę.

Dane poziomów przechowywane są w liście `LEVELS`, zawierającej współrzędne platform, monet, rejestrów i dronów. Umożliwia to łatwe dodawanie nowych poziomów.

2.3 Obsługa zdarzeń

W głównej pętli gry (`run`) wywoływane są metody `handle_events`, `update` i `draw`. Zdarzenia klawiatury i myszy są przechwytywane w `handle_events` i kierowane do odpowiednich przycisków lub bezpośrednio do logiki gry (np. klawisz I przełącza flagę). Przykład fragmentu kodu obsługującego zdarzenia w stanie gry:

```
elif self.state == "GAME":
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_i:
            self.interrupt_flag = 1 - self.interrupt_flag
            self.add_message(f"Flaga I = {self.interrupt_flag}")
        if event.key == pygame.K_ESCAPE:
            self.state = "MENU"
```

2.4 Mechanika ruchu i kolizji

Klasa `Player` implementuje prostą fizykę: prędkość w pionie `vel_y` jest modyfikowana przez grawitację, a skok powoduje nadanie prędkości ujemnej. Kolizje z platformami są sprawdzane osobno dla osi X i Y, co zapobiega przyklejaniu się do ścian. Fragment metody `check_collision`:

```
def check_collision(self, dx, dy, platforms):
    for platform in platforms:
        if self.rect.colliderect(platform.rect):
            if dx > 0:
                self.rect.right = platform.rect.left
            if dx < 0:
                self.rect.left = platform.rect.right
            if dy > 0:
                self.rect.bottom = platform.rect.top
                self.vel_y = 0
                self.on_ground = True
            if dy < 0:
                self.rect.top = platform.rect.bottom
                self.vel_y = 0
```

2.5 System monet i rejestrów

Monety i rejestry są usuwane z grupy sprite'ów po kolizji z graczem, a odpowiednia liczba monet jest dodawana do stanu gry. Rejestry mają przypisaną nazwę (AX, BX, CX, DX) i po zebraniu wyświetlają komunikat. Kod odpowiedzialny za zbieranie:

```
collected_coins = pygame.sprite.spritecollide(self.player, self.
    coins_group, True)
if collected_coins:
    self.coins += len(collected_coins)
    self.add_message(f"+{len(collected_coins)} monet")
```

2.6 System skórek i sklep

Skórki są wczytywane z plików PNG i skalowane do rozmiaru 40x80 pikseli. Stan odblokowania przechowywany jest w słowniku `unlocked_skins`. W sklepie wyświetlane są przyciski z miniaturkami; kliknięcie uruchamia metodę `buy_skin`, która sprawdza stan konta i odblokowuje skórkę. Po zakupie przyciski są odświeżane. Przykład tworzenia przycisku w sklepie:

```
thumb = pygame.transform.scale(self.skins[skin], (30, 60))
btn = Button(150, y, 300, 70, text, thumbnail=thumb,
             action=lambda s=skin: self.buy_skin(s))
```

2.7 System poziomów i odblokowywanie

Poziomy są numerowane od 0. Po ukończeniu poziomu (brak monet i rejestrów) wywoływana jest metoda `level_complete`, która odblokowuje następny poziom (jeśli istnieje) i odświeża listę przycisków w menu wyboru. Następnie stan gry wraca do `LEVEL_SELECT`.

2.8 Pasek ciekawostek

Ciekawostki są przechowywane w liście `FACTS`. Co 3 sekundy (60 FPS * 3) indeks jest zwiększany, a tekst zawijany za pomocą funkcji `wrap_text`, aby nie wychodził poza ekran. Fragment rysowania:

```
lines = wrap_text(f"       {fact}", self.small_font, SCREEN_WIDTH -
    40)
y = SCREEN_HEIGHT - bar_height + 10
for line in lines:
    fact_surf = self.small_font.render(line, True, (200, 200, 0))
    self.screen.blit(fact_surf, (20, y))
    y += 22
```

3 Dyskusja osiągniętych wyników

3.1 Zalety aplikacji

- **Edukacyjny charakter**: gra w przystępny sposób ilustruje pojęcia z architektury komputerów (rejestry, przerwania, flaga I, tryby pracy, licznik rozkazów). Pasek

ciekawostek dostarcza dodatkowej wiedzy.

- **Rozbudowany interfejs**: menu główne, wybór poziomów, sklep, wybór postaci – wszystko obsługiwane myszą. Personalizacja wyglądu zwiększa atrakcyjność.
- **Skalowalność**: dodanie nowych poziomów, skórek czy ciekawostek wymaga jedynie modyfikacji danych, bez ingerencji w logikę.
- **Estetyka i spójność graficzna**: wszystkie elementy graficzne zostały zaprojektowane w jednolitym stylu, co tworzy przyjemną dla oka całość.
- **Niezawodność**: kod jest odporny na brak plików graficznych – w razie problemów używane są zastępcze kolory.

3.2 Wady i ograniczenia

- **Ograniczona liczba poziomów**: obecnie trzy poziomy, choć łatwo dodać kolejne.
- **Brak dźwięku**: gra nie posiada efektów dźwiękowych ani muzyki, co mogłoby zwiększyć immersję.
- **Prosta fizyka**: grawitacja i skok są stałe, nie ma możliwości regulacji (np. podwójny skok).
- **Brak zapisu stanu gry**: po zamknięciu programu postępy (odblokowane skórki, monety) nie są trwale zapisywane.
- **Nieoptymalne zachowanie dronów**: drony poruszają się losowo, czasami utykają w ścianach lub wlatują w niedostępne miejsca.

3.3 Możliwości rozwoju

- Dodanie zapisu do pliku (np. JSON) stanu gry (monety, odblokowane skórki, poziomy).
- Implementacja efektów dźwiękowych (zbieranie monet, uderzenie drona, ukończenie poziomu).
- Więcej poziomów, przeciwników, przeszkód.
- Wprowadzenie trybu edytora poziomów.
- Rozbudowa systemu flag (np. flaga D wpływająca na kierunek skoku).

3.4 Podsumowanie

Projekt `CPU Quest` spełnia wszystkie wymagania postawione w zadaniu. Aplikacja posiada graficzny interfejs użytkownika, obsługuje interakcję za pomocą klawiatury i myszy, umożliwia konfigurację wyglądu (skórki) oraz wyświetla bieżący czas. Dzięki nawiązaniom do tematyki przedmiotu stanowi wartościowe narzędzie dydaktyczne, które może być wykorzystywane podczas zajęć z organizacji systemów komputerowych.